

Day08 课堂笔记

0. 复习和反馈

`os.rmdir('path')` :删除指定path的空目录，但是有道题是直接说删除path目录，不是应该排定是否为空目录吗

基础班考试是以什么形式，笔试 pycharm还是学生端的选择，题的难度怎么样。。。。

1 单选 多选

2 代码题

3

4 `while True: pass` 使用场景, 在书写代码的时候, 不确定循环一共执行多少次, 即不知道循环在什么时候结束, 具体结束的时间, 在运行代码的过程中才能确定, 在代码中进行 `if` 判断, 如果条件不满足, 就使用 `break`, 终止循环

5

6 `my_list = list()` # 创建列表类对象

7 `my_dict = dict()` # 创建字典类对象

8 `my_tuple = tuple()` # 创建元组类对象

9 -----

10 `class 类名(object):`

11 `def 函数名(self):`

12 `pass`

1. 类外部添加和获取对象属性

```

1 class Dog(object):
2     def play(self):
3         print('小狗快乐的拆家中.....')
4
5
6 # 创建对象
7 dog = Dog()
8 dog.play()
9
10 # 给对象添加属性 对象.属性名 = 属性值
11 dog.name = '大黄' # 给dog对象添加name属性, 属性值是 大黄
12 dog.age = 2 # 给dog对象添加age属性, 属性值是 2
13
14 # 获取对象的属性值 对象.属性名
15 print(dog.name)
16 print(dog.age)
17
18 # 修改属性值 和添加一样, 存在就是修改, 不存在, 就是添加
19 dog.age = 3 # age 属性已经存在, 所以是修改属性值
20 print(dog.age)
21
22 dog1 = Dog() # 新建一个对象 dog1
23 dog1.name = '小白'
24 print(dog1.name)

```

每个对象，会保存自己的属性值，不同对象的属性值之间没有关联

Process finished with exit code 0

小狗快乐的拆家中.....

大黄
2
3

小白

Process finished with **exit code 0**

2. 类内部操作属性

```
1 class Dog(object): self 就是一个形参的名字，可以写成其他的形参名，一般不修改这个名字，默认是 self
2     # self 作为类中方法的第一个形参，在通过对象调用方法的时候，不需要手动传递实参值，是python解释器
3     # 自动将调用该方法的对象传递给self，所以self这个形参代表的是对象
4     def play(self):
5         print(f'self: {id(self)}')
6         print(f'小狗 {self.name} 在快乐的拆家中.....')
7
8
9 # 创建对象
10 dog = Dog()
11 dog.name = '大黄'
12 print(f"dog : {id(dog)}")
13 dog.play()
14 print('-' * 30)
15 dog1 = Dog()
16 dog1.name = '小白'
17 print(f"dog1: {id(dog1)}")
18 dog1.play()
19
```

dog : 2234929865776
self: 2234929865776
小狗 大黄 在快乐的拆家中.....

dog1: 2234930129360
self: 2234930129360
小狗 小白 在快乐的拆家中.....

```
dog : 2234929865776
self: 2234929865776
小狗 大黄 在快乐的拆家中.....
-----
dog1: 2234930129360
self: 2234930129360
小狗 小白 在快乐的拆家中.....
```

3. 魔法方法

- 1 在 python 的类中,有一类方法,这类方法以`两个下划线开头`和`两个下划线结尾`,并且在`满足某个特定条件的情况下,会自动调用`. 这类方法,称为魔法方法
- 2
- 3 如何学习魔法方法:
- 4 1. 魔法方法在什么情况下会自动调用
- 5 2. 这个魔法方法有什么作用
- 6 3. 这个魔法方法有哪些注意事项

`__init__()` [掌握]

- 1 调用时机: 在创建对象之后,会立即调用.
- 2 作用:
 - 3 1. 用来给对象添加属性,给对象属性一个初始值(构造函数)
 - 4 2. 代码的业务需求,每创建一个对象,都需要执行的代码可以写在`\_\_init\_\_`中
- 5 注意点: 如果`\_\_init\_\_`方法中,有出了`self`之外的形参,那么在创建的对象的时候,需要给额外的形参传递实参值`类名(实参)`

```

1 class Dog(object):
2     def __init__(self): # self 是对象
3         print('我是__init__方法,我被调用了')
4         # 对象.属性名 = 属性值
5         self.name = '小狗'
6
7
8 # 创建对象
9 # Dog()
10 dog = Dog()
11 print(dog.name)
12 dog1 = Dog()
13 print(dog1.name)

```

__init__()

- 1 调用时机：在创建对象之后,会立即调用。
- 2 作用：
 1. 用来给对象添加属性,给对象属性一个初始值(构造函数)
 2. 代码的业务需求,每创建一个对象,都需要执行的代码可以写在 `\_\_init\_\_` 中

Run: 03-__init__方法 ×

C:\Develop\python38\python.exe C:/Users/nl/Desktop/43/day08/03-__init__方法.py

我是__init__方法,我被调用了

小狗

我是__init__方法,我被调用了

小狗

```

1 class Dog(object):
2     def __init__(self, name): # self 是对象
3         print('我是__init__方法,我被调用了')
4         # 对象.属性名 = 属性值
5         self.name = name
6
7
8 # 创建对象 类名(实参值)
9 dog = Dog('大黄')
10 print(dog.name)
11 dog1 = Dog('小白')
12 print(dog1.name)
13

```

Run: 04-书写带参数的init方法 ×

C:\Develop\python38\python.exe C:/Users/nl/Desktop/43/day

我是__init__方法,我被调用了

大黄

我是__init__方法,我被调用了

小白

__str__() [掌握]

- 1 调用时机:
- 2 1. `print(对象)`, 会自动调用 `__str__` 方法, 打印输出的结果是 `__str__` 方法的返回值
- 3 2. `str(对象)` 类型转换, 将自定义对象转换为字符串的时候, 会自动调用
- 4 应用:
- 5 1. 打印对象的时候, 输出一些属性信息
- 6 2. 需要将对象转换为字符串类型的时候
- 7 注意点:
- 8 `方法必须返回一个字符串`, 只有 `self` 一个参数

```
1 class Dog(object):
2     def __init__(self, name, age):
3         # 添加属性
4         self.name = name
5         self.age = age
6
7     def __str__(self): 时机代码中不写这行代码
8         print('我是__str__, 我被调用了...')
9         # 必须返回一个字符串
10        return f"小狗的名字是{self.name}, 年龄是{self.age}"
11
12
13 # 创建对象
14 dog = Dog('大黄', 2)
15 print(dog) # 没有定义 __str__ 方法, print(对象) 默认输出对象的引用地址
16
17 str_dog = str(dog) # 没有定义 __str__ 方法, 类型转换, 赋值的也是引用地址
18 print(str_dog)
19
20 我是__str__, 我被调用了...
21 小狗的名字是大黄, 年龄是2
22 我是__str__, 我被调用了...
23 小狗的名字是大黄, 年龄是2
24
25 Process finished with exit code 0
```

`__str__` [掌握]

- 1 调用时机:
- 2 1. `print(对象)`, 会自动调用 `__str__` 方法, 打印输出的结果是 `__str__` 方法的返回值
- 3 2. `str(对象)` 类型转换, 将自定义对象转换为字符串的时候, 会自动调用
- 4 应用:
- 5 1. 打印对象的时候, 输出一些属性信息
- 6 2. 需要将对象转换为字符串类型的时候
- 7 注意点:
- 8 `方法必须返回一个字符串`, 只有 `self` 一个参数

<__main__.Dog object at 0x0000023ADE636430>
<__main__.Dog object at 0x0000023ADE636430>

Process finished with exit code 0

激活

`__del__()` [理解]

- 1 析构函数
- 2 调用时机:

3 对象在内存中被销毁删除的时候(引用计数为 0)会自动调用 `__del__` 方法

4 1. 程序代码运行结束,在程序运行过程中,创建的所有对象和变量都会被删除销毁

5 2. 使用 `del 变量`, 将这个对象的引用计数变为 0.会自动调用 `__del__` 方法

6 应用场景:

7 对象被删除销毁的时候,要书写的代码可以写在 `__del__` 中.一般很少使用

8

9 引用计数: 是 python 内存管理的一种机制, 是指一块内存,有多少个变量在引用,

10 1. 当一个变量,引用一块内存的时候,引用计数加 1

11 2. 当删除一个变量,或者这个变量不再引用这块内存.引用计数减 1

12 3. 当内存的引用计数变为 0 的时候,这块内存被删除,内存中的数据被销毁

13

14 `my_list = [1, 2] # 1`

15 `my_list1 = my_list # 2`

16 `del my_list # 1`

17 `del my_list1 # 0`

18

```

1 class Dog(object):
2     def __init__(self, name, age):
3         # 添加属性
4         self.name = name
5         self.age = age
6
7     def __str__(self): ...
10
11     def __del__(self):
12         print(f'我是__del__ 方法,我被调用了, {self.name}被销毁了.....')
13
14
15 # 创建一个对象
16 # dog = Dog('大黄', 2)
17 # dog1 = Dog('小白', 1)
18
19 dog = Dog('小花', 3) # 小花 引用计数为1
20 dog2 = dog # 小花 引用计数2
21 print('第一次删除之前')
22 del dog # dog 变量不能使用, 小花对象引用计数 1
23 print('第一次删除之后')
24 print('第二次删除之前')
25 del dog2 # dog2变量不能使用, 小花对象的引用计数为 0, 会立即__del__ 方法
26 print('第二次删除之后')

```

```

1 析构函数
2 调用时机:
3 对象在内存中被销毁删除的时候(引用计数为 0)会自动调用 __del__ 方法
4 1. 程序代码运行结束,在程序运行过程中,创建的所有对象和变量都会被删除
   销毁
5 2. 使用 'del 变量', 将这个对象的引用计数变为 0.会自动调用
   __del__ 方法
6 应用场景:
7 对象被删除销毁的时候,要书写的代码可以写在 '__del__'中.一般很少使用

```

```

第一次删除之前
第一次删除之后
第二次删除之前
我是__del__ 方法,我被调用了, 小花被销毁了.....
第二次删除之后

```

激活Wind

案例: 烤地瓜

- 1 封装的小套路:
- 2 1. 根据文字的描述信息,确定对象,对象有什么,就是属性
- 3 2. 根据文字的描述信息, 对象能干什么,就是方法
- 4 3. 根据文字的描述信息, 确定方法怎么书写

```

1 类名：地瓜类 Potato
2 属性：
3     状态 status='生的'
4     烧烤总时间 total_time = 0
5 方法：
6     def cook(self, 烧烤时间):
7         计算烧烤的总时间
8         修改地瓜的状态的
9         pass
10    输出信息 __str__()
11    定义属性 __init__()

```

day08 07-烤地瓜.py

06-__del__方法.py

07-烤地瓜.py

```

1 class Potato(object):
2     def __init__(self):
3         self.status = '生的'
4         self.total_time = 0
5
6     def cook(self, time):
7         # 计算总时间
8         self.total_time += time
9         # 修改地瓜的状态
10        if self.total_time < 3:
11            self.status = '生的'
12        elif self.total_time < 6:
13            self.status = '半生不熟的'
14        elif self.total_time < 8:
15            self.status = '熟了'
16        else:
17            self.status = '烤糊了'
18
19    def __str__(self):
20        return f"地瓜的状态<<{self.status}>>, 烧烤总时间为<{self.total_time}>"

```

烤地瓜规则:

- 1.地瓜有自己的状态, 默认是生的, 地瓜可以进行烧烤
- 2.地瓜有自己烧烤的总时间, 由每次烧烤的时间累加得出
- 3.地瓜烧烤时, 需要提供本次烧烤的时间
- 4.地瓜烧烤时, 地瓜状态随着烧烤总时间的变化而改变: [0, 3) 生的、[3, 6) 半生不熟了、>=8 烤糊了
- 5.输出地瓜信息时, 可以显示地瓜的状态和烧烤的总时间

1 类名：地瓜类 Potato

2 属性：

3 状态 status='生的'

4 烧烤总时间 total_time = 0

5 方法：

6 def cook(self, 烧烤时间):

7 计算烧烤的总时间

8 修改地瓜的状态的

9 pass

10 输出信息 __str__()

11 定义属性 __init__()

23 # 创建对象

24 potato = Potato()

25 print(potato)

26 potato.cook(4)

27 print(potato)

28 potato.cook(3)

29 print(potato)

30

地瓜的状态<<生的>>, 烧烤总时间为<0>

地瓜的状态<<半生不熟的>>, 烧烤总时间为<4>

地瓜的状态<<熟了>>, 烧烤总时间为<7>

Process finished with exit code 0

激活 Windows
转到“设置”应用以激活 Windows。

烤地瓜-调料版

- 1 属性：
- 2 调料: name_list = []
- 3 方法：
- 4 添加调料 add()

```
2 def __init__(self):
3     self.status = '生的'
4     self.total_time = 0
5     self.name_list = [] # 保存调料的

20 def __str__(self):
21     # buf_list = str(self.name_list) # str([]) ==> '[]'
22     # buf_list = buf_list.replace('[', '')
23     # 字符串.join(列表), 将字符串添加到列表中的每个元素之间, 组成新的字符串
24     buf = ','.join(self.name_list)
25     if self.name_list:
26         return f"地瓜的状态<<{self.status}>>, 烧烤总时间为<{self.total_time}>, 调料有: {buf}"
27     else:
28         return f"地瓜的状态<<{self.status}>>, 烧烤总时间为<{self.total_time}>, 还没有添加调料"
29
30 def add(self, name):
31     self.name_list.append(name)
```

Potato > cook() > if self.total_time < 3

Run: 08-烤地瓜-调料版

C:\Develop\python38\python.exe C:/Users/nl/Desktop/43/day08/08-烤地瓜-调料版.py

地瓜的状态<<生的>>, 烧烤总时间为<0>, 还没有添加调料

地瓜的状态<<半生不熟的>>, 烧烤总时间为<4>, 调料有: 油, 辣椒面

地瓜的状态<<熟了>>, 烧烤总时间为<7>, 调料有: 油, 辣椒面, 孜然

案例: 搬家具

- 1 类名: 家具类 Furniture
- 2 属性:
- 3 类型 name
- 4 面积 area
- 5 方法:
- 6 输出家具信息 __str__
- 7 定义属性 __init__

```
8  -----
9  类名：房子类 House
10 属性：
11      地址 address
12      面积 h_area
13      家具列表 furniture_list = []
14 方法：
15      添加家具 add_furniture()
16      输出房子信息 __str__
17      定义属性 __init__
18
```

```
13 # 定义房子类
14 class House(object):
15     def __init__(self, address, area):
16         self.address = address
17         self.h_area = area
18         self.furniture_list = []
19         self.free_area = area # 房子的剩余面积
20
21     def add_furniture(self, obj_furniture):
22         """ 添加家具 obj_furniture: 家具类的对象 """
23         if self.free_area > obj_furniture.area:
24             self.furniture_list.append(obj_furniture)
25             # 修改剩余面积
26             self.free_area -= obj_furniture.area
27             print(f'家具<{obj_furniture.name}>添加成功')
28         else:
29             print('添加失败, 换个大房子吧')
30
31     def __str__(self):
32         return f"房子的地址为<{self.address}>, 占地面积为<{self.h_area}>, 剩余面积为{self.free_area}"
33
34 家具的类型<豪华双人床>, 占地面积<15>平
35 房子的地址为<意大利农场>, 占地面积为<100>, 剩余面积为100
36 家具<豪华双人床>添加成功
37 房子的地址为<意大利农场>, 占地面积为<100>, 剩余面积为85
```

```
1 # 定义家具类 Furniture 类
2 class Furniture(object):
3     def __init__(self, name, area):
4         # 类型
5         self.name = name
6         # 面积
7         self.area = area
8
9     def __str__(self):
10        return f'家具的类型<{self.name}>, 占地面积<{self.area}>平'
11
12 # 创建家具对象
13 bed = Furniture('豪华双人床', 15)
14 print(bed)
15
16 # 创建一个房子类对象
17 house = House('意大利农场', 100)
18 print(house)
19 house.add_furniture(bed)
20 print(house)
```

激活 Windows
转到“设置”以激活 Windows。

```
31 def __str__(self):
32     # 自定义家具类, 将该类的对象添加到列表中(容器), 直接打印列表, 显示的是 自定义对象的引用地址
33     # [家具对象, 家具对象, ... ] ---> [家具类型, 家具类型, ....]
34     if self.furniture_list:
35         buf_list = [obj.name for obj in self.furniture_list]
36         return f"房子的地址为<{self.address}>, 占地面积为<{self.h_area}>, 剩余面积为{self.free_area},
37             f"家具具有<{' '.join(buf_list)}>"
38     else:
39         return f"房子的地址为<{self.address}>, 占地面积为<{self.h_area}>, 剩余面积为{self.free_area},
40             f"还没有购买家具"
41
42
43
```

House > _str_() > if self.furniture_list

Run: 11-搬家具-打印家具信息

C:\Develo\python38\python.exe C:/Users/n1/Desktop/43/day08/11-搬家具-打印家具信息.py

家具的类型<豪华双人床>, 占地面积<15>平

房子的地址为<意大利农场>, 占地面积为<100>, 剩余面积为100, 还没有购买家具

家具<豪华双人床>添加成功

房子的地址为<意大利农场>, 占地面积为<100>, 剩余面积为85, 家具具有<豪华双人床>